

IEEE GBPIET Student Branch Backend

Automated Certificate Generation, Event Management, Department Reporting, Support Ticketing & Admin Analytics System

Overview

The **IEEE GBPIET Backend** is a modular RESTful API built on **Node.js (Express 5)** and **MongoDB (Mongoose)** designed to power the official web platform of the **IEEE Student Branch at Govind Ballabh Pant Institute of Engineering & Technology (GBPIET)**.

It provides automated digital certificate generation and email dispatching, media uploads for departmental posts and upcoming events, ticket-based public inquiry management, and real-time analytical dashboard reporting for administrators.

Key Features

- **Automated Certificate Generation & Email Delivery:**
 - Headless Chromium PDF rendering powered by **Puppeteer**.
 - Dynamic responsive HTML certificate templates (Winner & Participation).
 - Transactional email delivery with PDF attachments via **Resend API**.
 - Public participant application flow with admin review (Approve/Reject) or direct admin issuance.
- **Media Upload Pipeline:**
 - Zero-disk memory storage with **Multer**.
 - Direct buffer streaming to **Cloudinary** cloud storage.
 - Automatic deletion of previous Cloudinary assets on post update or deletion to eliminate orphaned assets.
 - File size limits (5MB) and MIME-type validation ([jpeg](#), [png](#), [webp](#), [jpg](#)).
- **Secure Authentication & RBAC:**
 - Stateless JSON Web Token (**JWT**) authentication via **Authorization: Bearer <token>** or HTTP cookies.
 - Password hashing with **Bcrypt**.
 - Time-limited, single-use 6-digit OTP password reset workflow with cryptographic hashing and MongoDB TTL expiry.
 - Rate limiting on sensitive endpoints (e.g. max 10 login requests per 15 minutes).
- **Department & Event Publications:**
 - Department-specific activity logging categorized by branch ([CSE](#), [AIML](#), [EE](#), [ECE](#), [BT](#)).
 - Upcoming events tracking with registration deadlines and overview details.
- **Support & Ticket Resolution:**
 - Public contact submission generating unique alphanumeric ticket IDs ([TKT...](#)).
 - Admin management for reviewing, rejecting, or closing/solving tickets.
- **Analytics & Admin Dashboard:**
 - Aggregated metrics across department publications, recent events, support resolution status, and certificate issuance statistics.

Technology Stack

Layer	Technology	Purpose
Runtime	Node.js (ES Modules)	High-performance asynchronous JavaScript runtime
Framework	Express 5	Web routing and REST API framework
Database	MongoDB via Mongoose 9	Document database and object data modeling (ODM)

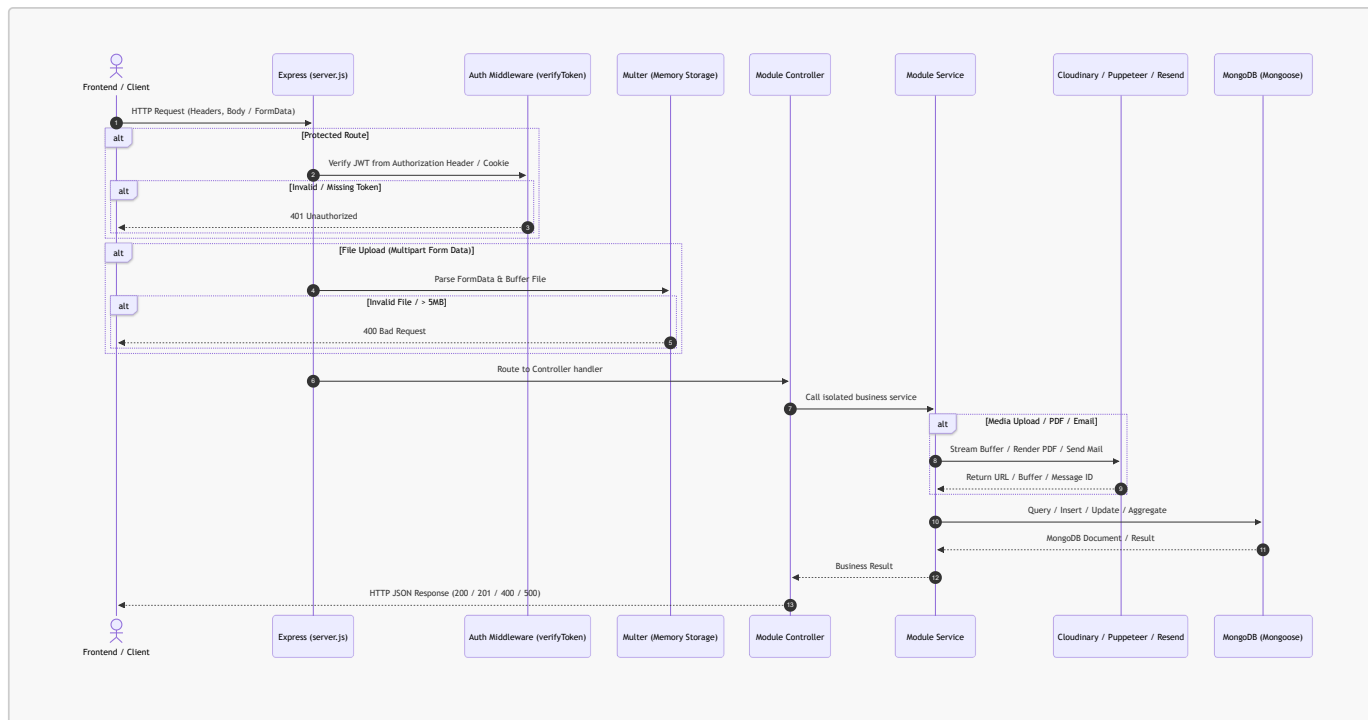
Layer	Technology	Purpose
Authentication	jsonwebtoken & bcrypt	JWT signing/verification and password hashing
Rate Limiting	express-rate-limit	Protection against brute-force attacks
Media Management	Cloudinary & Multer	Multipart file handling and cloud media hosting
PDF Generation	Puppeteer	Headless browser for printing HTML templates to PDF buffers
Email Service	Resend	Transactional email delivery API
Dev Tooling	Nodemon & dotenv	Live reloading and environment configuration

 System & Server Architecture

The application adopts a **Modular Controller-Service-Repository Pattern** ensuring loose coupling, high maintainability, and clean separation of concerns:

```
IEEE_backend/
├── server.js                # HTTP Server bootstrap & central route registration
├── src/
│   ├── config/             # Infrastructure & third-party client configurations
│   │   ├── cloudinary.js   # Cloudinary SDK client & upload/delete helpers
│   │   ├── mongodb.js      # Mongoose database connection
│   │   ├── multer.js       # Multer memory storage & validation middleware
│   │   └── resend.js       # Resend email client instance
│   ├── middleware/        # Express middleware
│   │   ├── auth.middleware.js # JWT verification middleware (Bearer token / cookies)
│   │   └── ratelimiter.js  # Rate limiter configurations (Login protection)
│   ├── models/            # Mongoose database schemas & models
│   │   ├── DepartmentPost.js # Departmental post schema
│   │   ├── certificate.js   # Certificate application & issuance schema
│   │   ├── contactUs.js     # Support tickets schema
│   │   ├── passwordReset.js # OTP password reset schema (TTL indexed)
│   │   ├── upcomingEvents.js # Upcoming events schema
│   │   └── user.js          # Admin user credentials schema
│   ├── modules/           # Feature modules (Route -> Controller -> Services)
│   │   ├── Support/        # Support ticket inquiry & management
│   │   ├── UpcomingEvent/  # Upcoming events publishing
│   │   ├── auth/           # Admin login, logout & OTP password recovery
│   │   ├── certificate/    # Certificate application, review & generation
│   │   ├── dashboard/     # Aggregated administrative analytics
│   │   └── department/     # Department activities and report logs
│   ├── templates/         # Dynamic HTML templates
│   │   ├── CertificateTemplate.js # Standard certificate HTML layout
│   │   ├── WinnerCertificate.js  # Winner certificate HTML layout (1st/2nd/3rd)
│   │   └── emailtemplate.js     # Transactional email HTML layout
│   └── utils/             # Shared utility functions
│       ├── EventsIDgenerator.js # Event post ID generator
│       ├── GeneratePostId.js    # Department post ID generator
│       ├── OtpGenerator.js      # Cryptographic 6-digit OTP generator
│       ├── certificateIDgenerator.js # Unique certificate ID generator
│       ├── generateCertificatePDF.js # Puppeteer PDF rendering engine
│       ├── jwt.sign.js          # JWT creation helper
│       ├── jwt.verify.js        # JWT verification helper
│       └── sendCertificateEmail.js # Resend email attachment dispatcher
```

Request Lifecycle Flow



Database Models & Schemas

1. User (`src/models/user.js`)

Stores administrator accounts.

- `email` (*String, required, unique, lowercase, immutable*): Admin email (`ieee@gbpiet.ac.in`).
- `hashedPassword` (*String, required, select: false*): Bcrypt-hashed password.
- `timestamps`: `createdAt`, `updatedAt`.

2. Certificate (`src/models/certificate.js`)

Tracks certificates issued or requested.

- `name` (*String, required, trim*): Recipient's full name.
- `email` (*String, required, lowercase, trim*): Recipient's email address.
- `branch` (*String, required*): Academic branch of the participant.
- `eventName` (*String, required*): Name of the event attended.
- `date` (*String, required*): Date of the event.
- `position` (*String, enum: ["1st", "2nd", "3rd"], default: null*): Position secured (if competitive event).
- `certificateId` (*String, required, unique*): Alphanumeric identifier formatted e.g., `IEEE-CERT-....`
- `status` (*String, enum: ["pending", "approved", "rejected"], default: "pending"*).
- `timestamps`: `createdAt`, `updatedAt`.

3. DepartmentPost (`src/models/DepartmentPost.js`)

Logs activity reports and news for academic departments.

- `id / postId` (*String, required, unique, index*): Formatted unique ID (e.g. `DEP-CSE-2026...`).
- `title` (*String, required, trim*): Post headline.
- `category` (*String, required, trim*): Event category (e.g. Workshop, Seminar, Hackathon).
- `branch` (*String, required, enum: ["CSE", "AIML", "EE", "ECE", "BT"], index*): Department branch.
- `date` (*String, required*): Event date.
- `time` (*String, trim*): Time schedule.
- `venue` (*String, required*): Event location/hall.

- `organizedBy` (*String, required*): Organizers / Chapter.
- `reportAuthor` (*String, trim*): Name of person authoring the report.
- `overview` (*String, required*): Brief executive summary.
- `description` (*String, required*): Full detailed narrative report.
- `keyDiscussion` (*Array of Strings*): Highlights / bullet points.
- `studentsPresent` (*Array of Strings*): Key attendee / volunteer names.
- `image` (*Object*):
 - `url` (*String*): Cloudinary secure URL.
 - `publicId` (*String*): Cloudinary asset ID.
- `timestamps`: `createdAt`, `updatedAt`.

4. `UpcomingEvent` (`src/models/upcomingEvents.js`)

Publishes upcoming IEEE branch events and workshops.

- `postId` (*String, required, unique, index*): Unique event ID.
- `eventName` (*String, required, trim*): Name of the event.
- `title` (*String, required, trim*): Promotional headline.
- `date` (*Date, required*): Event scheduled date.
- `lastDate` (*Date, required*): Registration deadline date.
- `overview` (*String, required, trim*): Description and prerequisites.
- `image` (*Object*):
 - `url` (*String*): Cloudinary banner URL.
 - `publicId` (*String*): Cloudinary asset ID.
- `timestamps`: `createdAt`, `updatedAt`.

5. `Ticket / ContactUs` (`src/models/contactUs.js`)

Handles public queries submitted via the contact form.

- `ticketId` (*String, required, unique, index*): Unique ticket code (e.g. `TKT4A9F21B0`).
- `name` (*String, required, trim*): Sender's name.
- `email` (*String, required, trim, lowercase*): Sender's email.
- `subject` (*String, required, trim*): Inquiry subject.
- `description` (*String, required, trim*): Detailed inquiry message.
- `solvedStatus` (*String, enum: ["pending", "rejected", "solved"], default: "pending", index*).
- `timestamps`: `createdAt`, `updatedAt`.

6. `PasswordReset` (`src/models/passwordReset.js`)

Manages transient OTP verifications for admin password resets.

- `email` (*String, required, enum: ["ieee@gbpiet.ac.in"]*): Restricted to authorized admin email.
- `otpHash` (*String, required*): Cryptographic hash of the 6-digit OTP.
- `expiresAt` (*Date, required*): Expiration timestamp.
- `attempts` (*Number, default: 0*): Count of failed entry attempts.
- **TTL Index**: MongoDB automatically purges documents when `expiresAt` is reached (`expireAfterSeconds: 0`).

Environment Variables

Create a `.env` file in the root directory by copying `.env.example`:

```
cp .env.example .env
```

Variable	Required	Description	Example
PORT	Optional	Port for Express server to bind to (defaults to 5000)	5000
MONGODB_URI	Yes	MongoDB connection string (Atlas or local)	mongodb+srv://user:pass@cluster.mongodb.net/ieee_backend
JWT_SECRET	Yes	Secret key for signing and verifying JWT tokens	super_secret_jwt_random_key_here
RESEND_API_KEY	Yes	Resend API key for automated email delivery	re_1234567890abcdef
EMAIL_FROM	Optional	Verified sender email address in Resend	"IEEE GBPIET <noreply@appnests.in>"
CLOUDINARY_CLOUD_NAME	Yes	Cloudinary Cloud Name	ieee-gbpiet
CLOUDINARY_API_KEY	Yes	Cloudinary API Key	123456789012345
CLOUDINARY_API_SECRET	Yes	Cloudinary API Secret	abcdefghijklmnopqrstuvwxyz123
FRONTEND_URL	Optional	Allowed CORS origin for frontend client	http://localhost:3000
CERTIFICATE_BASE_URL	Optional	Base URL for certificate QR / verification link	http://localhost:3000/verify
CERTIFICATE_LOGO_URL	Optional	Direct image URL for IEEE logo in	https://res.cloudinary.com/.../ieee_logo.png

Variable	Required	Description	Example
		certificate	
		PDF	

API Modules Summary

 For complete API specifications with request/response samples, headers, status codes, and query parameters, see [Apis.md](file:///d:/IEEE_backend/Apis.md).

Module	Base Path	Endpoints Count	Description
Health Check	/health	1	Server health & uptime status check
Authentication	/api/v1/auth	5	Admin login, logout, OTP generation, verification & password reset
Certificates	/api/v1/certificate	5	Certificate applications, approval, direct admin issuance & PDF mailing
Departments	/api/v1/department	5	CRUD operations for department news/reports with Cloudinary image upload
Upcoming Events	/api/v1/upcomingevents	5	CRUD operations for upcoming events with Cloudinary banner upload
Support	/api/v1/support	5	Public inquiry submission & admin ticket management (close/reject/view)
Dashboard	/api/v1/dashboard	4	Aggregated admin analytics for departments, events, tickets & certificates
Total		30	

Installation & Local Development

Prerequisites

- **Node.js:** v18.0.0 or higher
- **npm:** v9.0.0 or higher
- **MongoDB:** Active MongoDB database URI (Atlas or local instance)
- **Cloudinary Account:** Cloud name, API key, and API secret
- **Resend Account:** API key and verified sending domain

1. Clone & Install Dependencies

```
git clone https://github.com/chandankoranga02/IEEE_backend.git
cd IEEE_backend
npm install
```

2. Configure Environment

```
cp .env.example .env
# Edit .env with your credentials
```

3. Start Development Server

```
npm run dev
```

The server will start on <http://localhost:5000>.

4. Verify Health Check

```
curl http://localhost:5000/health
```

Response:

```
{
  "success": true,
  "message": "healthy"
}
```

Security Best Practices Implemented

1. **JWT Verification** (`auth.middleware.js`): All administrative and write routes enforce authenticated access.
2. **Rate Limiting** (`loginLimiter`): Safeguards the `/api/v1/auth/login` endpoint against brute-force password guessing.
3. **Memory Storage for Uploads**: Files are received into server RAM without persistent disk writes, preventing server storage leaks.
4. **Cloudinary Asset Synchronization**: When modifying or deleting posts, old Cloudinary assets are destroyed via API.
5. **Atomic Certificate Rollback**: If PDF generation or Resend delivery fails, database mutations are safely reverted.
6. **Encrypted Passwords**: Passwords stored strictly as Bcrypt hashes with salt rounds.

License

This project is licensed under the [MIT License](#). Developed for **IEEE Student Branch GBPIET**.